

Numerical Linear Algebra

Weekly assignment #2

1. Write a Julia code that starts with something like

```
using Random
using LinearAlgebra
m = 10; n = 7;
A = randn(rng,Float64,m,n);
b = randn(rng,Float64,m,1);
Q, R = qr(A)
```

The goal is to find x in $\mathbb{R}^n$ such that x realizes the minimum of $\min_{x \in \mathbb{R}^n} \|b - Ax\|_2$ using the computed factors Q and R . Check that the answer x is correct. Note that Q might never be explicitly computed and you can manipulate it with $Q' * x$ for example, and that will return what you expect.

2. Write a Julia code that starts with something like

```
using Random
using LinearAlgebra
rng = MersenneTwister()
m = 7; n = 10;
A = randn(rng,Float64,m,n);
b = randn(rng,Float64,m,1);
```

The goal is to find x in $\mathbb{R}^n$ such that x is the x of minimum 2-norm such that $Ax = b$. using a QR factorization and the computed factors Q and R . In other words, x realizes $\min_{x \in \mathbb{R}^n, Ax=b} \|x\|_2$. Check that the answer x is correct.

3. Write a Julia code that starts with something like

```
using Random
using LinearAlgebra
m = 10; n = 7; k = 4;
A = randn(rng,Float64,m,k) * randn(rng,Float64,k,n);
b = randn(rng,Float64,m,1);
U, S, V = svd(A)
```

Note that A is constructed such that the rank of A is $k = 4$. The goal is to find x in $\mathbb{R}^n$ such that x is the vector of minimum norm that realizes the minimum of $\min_{x \in \mathbb{R}^n} \|b - Ax\|_2$ using the computed factors U , S and V . Check that the answer x is correct.

Note: x can also be computed with a “complete orthogonal decomposition”.

4. Here is an algorithm to compute a QR factorization of A .

```

using Random
using LinearAlgebra
rng = MersenneTwister()
m = 10;
n = 7;
A = randn(rng,Float64,m,n);
Q = copy(A)
R = randn(rng,Float64,n,n);
BLAS.syrk!( 'U', 'T', 1.0, Q, 0., R )
LAPACK.potrf!( 'U', R )
BLAS.trsm!( 'R', 'U', 'N', 'N', 1.0, R, Q )

```

Check that this algorithm works. (I.e. check that the computed Q and R factors are QR factors of a QR factorization of A .) Explain how this three-line algorithm works and why it works. (By typing your explanations in your file as some "comments".) Compute the number of operations of this algorithm as a function of m and n . Change the condition number of the input matrix A and observe how $\|I - Q^T Q\|_2$ depends on the condition number. What do you think is the relation of $\|I - Q^T Q\|_2$ as a function of the condition number? Time this code for a matrix $m = 5,000$ and $n=1,000$. Compute the performance in GFLOP/sec. Compare with built-in julia QR. (You can leave Q in the "Householder vector" form.) Or you can call LAPACK directly with

```

tau = randn(rng,Float64,n);
LAPACK.geqrf!( A, tau )

```

5. We start with

```

m = 10; n = 7; A = randn(rng,Float64,m,n);
u = randn(rng,Float64,m,1);
v = randn(rng,Float64,n,1);
Q, R = qr(A)
Q = Matrix(Q)

```

We now consider the matrix $B = A + uv^T$. B is a rank-1 update of A . Design an algorithm to find the QR factorization of B in $\mathcal{O}(n^2)$ operations by updating the Q and R factors of A . Implement your algorithm in Julia and verify the correctness.